

LAPORAN PRATIKUM STRUKTUR DATA

Doubly Linked List



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Wahyudi. DR, S.T.M.T

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2026

KATA PENGANTAR

Puji dan syukur kita panjatkan kehadiran Allah yang Maha Esa karena telah memberikan nikmat sehat jasmani dan rohani sehingga saya bisa menyelesaikan laporan ini. Di dalam laporan ini, terdapat hasil praktikum, pembahasan serta kesimpulan yang saya susun secara sistematis. Saya sebagai penulis sekaligus penyusun laporan ini sangat berterimakasih kepada Bapak Wahyudi selaku dosen dalam mata kuliah praktikum Struktur Data Departemen Informatika, Universitas Andalas, yang telah menuntun dan mengajarkan saya mata kuliah ini.

Dalam penulisan laporan praktikum ini, saya mendapatkan suatu pelajaran baik serta mendapatkan pengetahuan tentang praktikum tersebut. Saya sebagai penulis dalam laporan ini masih dalam proses belajar, baik dalam tata penulisan laporan yang benar maupun dalam memahami penyajian laporan yang sesuai kaidah. Untuk itu saya berharap mendapatkan saran-saran yang membangun sehingga dapat meningkatkan pengetahuan saya dan dapat membuat isi laporan yang lebih baik, mendidik dan dapat dipertanggungjawabkan.

Padang, 2026

Shofwatul Hasanah Sarma

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I.....	3
PENDAHULUAN	Error! Bookmark not defined.
1.1 Latar Belakang.....	3
1.2 Tujuan Praktikum	3
1.3 Manfaat Praktikum	4
BAB II.....	5
PEMBAHASAN	5
2.1 Pengertian Single Linked List	5
2.2 Praktikum	5
BAB III	31
KESIMPULAN.....	31
3.1 Ringkasan Hasil Praktikum	31
3.2 Saran Pengembangan.....	32
DAFTAR PUSTAKA	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data yang efisien merupakan hal yang sangat penting untuk membangun sebuah sistem yang terstruktur dan mudah dikembangkan. Salah satu struktur data yang banyak digunakan adalah linked list, khususnya Doubly Linked List. Doubly Linked List merupakan struktur data dinamis yang setiap node memiliki dua pointer, yaitu pointer ke node berikutnya (next) dan pointer ke node sebelumnya (prev). Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan dari dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan.

Dibandingkan dengan struktur data array yang bersifat statis, Doubly Linked List memiliki kelebihan dalam proses penambahan dan penghapusan data karena tidak perlu menggeser elemen lain. Hal ini membuat Doubly Linked List lebih fleksibel dalam pengelolaan data yang dinamis. Oleh karena itu, pemahaman mengenai konsep dan implementasi Doubly Linked List sangat penting bagi mahasiswa dalam mempelajari struktur data dan algoritma

Melalui praktikum ini, mahasiswa diharapkan mampu memahami cara kerja Doubly Linked List, mulai dari proses pembuatan node, penambahan data, penghapusan data, hingga penelusuran data dari dua arah menggunakan bahasa pemrograman Java.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar Doubly Linked List.
2. Memahami struktur node pada Doubly Linked List yang memiliki pointer next dan prev.
3. Mampu mengimplementasikan Doubly Linked List menggunakan bahasa pemrograman Java.
4. Mengetahui cara melakukan operasi dasar pada Doubly Linked List, seperti:
 - o Menambah node (insert)
 - o Menghapus node (delete)
 - o Menelusuri node maju dan mundur (traversal)
5. Melatih kemampuan logika pemrograman dalam pengelolaan struktur data dinamis.
6. Memahami penggunaan pointer atau referensi antar node pada linked list.

1.3 Manfaat Praktikum

Adapun manfaat dari praktikum ini adalah sebagai berikut:

1. Mahasiswa dapat memahami konsep struktur data dinamis menggunakan Doubly Linked List.
2. Mahasiswa mampu memahami hubungan antar node menggunakan pointer next dan prev.
3. Meningkatkan kemampuan mahasiswa dalam membuat program manipulasi data secara dinamis.
4. Membantu mahasiswa memahami proses traversal data dari dua arah.
5. Melatih kemampuan berpikir logis dan sistematis dalam pemrograman.
6. Menjadi dasar dalam mempelajari struktur data yang lebih kompleks seperti stack, queue, dan tree

BAB II

PEMBAHASAN

2.1 Pengertian

Doubly Linked List adalah struktur data linear yang terdiri dari kumpulan node, di mana setiap node memiliki tiga bagian, yaitu data, pointer ke node berikutnya (next), dan pointer ke node sebelumnya (prev). Struktur ini memungkinkan proses traversal dilakukan secara dua arah, yaitu dari head ke tail maupun dari tail ke head.

Pada Doubly Linked List, setiap node saling terhubung melalui pointer next dan prev sehingga proses penambahan dan penghapusan node menjadi lebih mudah dibandingkan Singly Linked List. Doubly Linked List banyak digunakan dalam pengembangan aplikasi yang membutuhkan navigasi dua arah, seperti playlist musik, fitur undo-redo, dan browser history.

2.2 Praktikum

⇒ Class HapusDLL_2511533015

A. Kode program

```
package pekan6_2511533015;

public class HapusDLL_2511533015 {
    //fungsi menghapus node awal
    public static NodeDLL_2511533015 delHead_3015 (NodeDLL_2511533015 head_3015) {
        if (head_3015==null) {
            return null;}
        NodeDLL_2511533015 temp = head_3015;
        head_3015 = head_3015.next_3015;
        if (head_3015 != null) {
            head_3015.prev_3015 = null; }
        return head_3015;
    }
}
```

```

//fungsi menghapus di akhir
public static NodeDLL_2511533015 delLast_3015 (NodeDLL_2511533015 head_3015) {
    if (head_3015 == null) {
        return null;
    }
    if (head_3015.next_3015 == null) {
        return null;
    }
    NodeDLL_2511533015 curr_3015 = head_3015;
    while (curr_3015.next_3015 != null) {
        curr_3015 = curr_3015.next_3015;
    }
    //update pointer previous node
    if (curr_3015.prev_3015 != null) {
        curr_3015.prev_3015.next_3015 = null;
    }
    return head_3015;
}

//fungsi menghapus node posisi tertentu
public static NodeDLL_2511533015 delPos_3015 (NodeDLL_2511533015 head_3015, int pos) {
    //jika DLL kosong
    if (head_3015 == null) {
        return head_3015;
    }
    NodeDLL_2511533015 curr_3015 = head_3015;
    //telusuri sampai ke node yang akan dihapus
    for (int i = 1; curr_3015 != null && i < pos; ++i) {
        curr_3015 = curr_3015.next_3015;
    }
    //jika posisi tidak ditemukan
    if (curr_3015 == null) {
        return head_3015;
    }
    //update pointer
    if (curr_3015.prev_3015 != null) {
        curr_3015.prev_3015.next_3015 = curr_3015.next_3015;
    }
}

```

```

        if (curr_3015.next_3015 != null) {
            curr_3015.next_3015.prev_3015 = curr_3015.prev_3015;
        }
        //jika yang dihapus head
        if (head_3015 == curr_3015) {
            head_3015 = curr_3015.next_3015;
        }
        return head_3015;
    }

    //fungsi mencetak DLL
    public static void printList_3015 (NodeDLL_2511533015 head_3015) {
        NodeDLL_2511533015 curr_3015 = head_3015;
        while (curr_3015 != null) {
            System.out.print(curr_3015.data_3015 + "<-> ");
            curr_3015 = curr_3015.next_3015;
        }
        System.out.println();
    }

    public static void main (String [] args) {
        //buat sebuah DLL
        NodeDLL_2511533015 head_3015 = new NodeDLL_2511533015 (1);
        head_3015.next_3015 = new NodeDLL_2511533015 (2);
        head_3015.next_3015.prev_3015 = head_3015;
        head_3015.next_3015.next_3015 = new NodeDLL_2511533015 (3);
        head_3015.next_3015.next_3015.prev_3015 = head_3015.next_3015;
        head_3015.next_3015.next_3015.next_3015 = new NodeDLL_2511533015 (4);
        head_3015.next_3015.next_3015.next_3015.prev_3015 =
head_3015.next_3015.next_3015;
        head_3015.next_3015.next_3015.next_3015.next_3015 = new NodeDLL_2511533015
(5);
        head_3015.next_3015.next_3015.next_3015.next_3015.prev_3015 =
head_3015.next_3015.next_3015.next_3015;

        System.out.print("DLL Awal: ");
        printList_3015 (head_3015);
    }

```



```

        System.out.print ("Setelah head dihapus: ");
        head_3015 = delHead_3015 (head_3015);
        printList_3015 (head_3015);

        System.out.print ("Setelah node terakhir dihapus: ");
        head_3015 = delLast_3015 (head_3015);
        printList_3015 (head_3015);

        System.out.print ("menghapus node ke 2: ");
        head_3015 = delPos_3015 (head_3015, 2);
        printList_3015 (head_3015);
    }
}

```

B. Penjelasan

1. Package dan class

```

Package pekan6_2511533015;
Public class HapusDLL_2511533015 {

```

- Package pekan6_2511533015
➔ Menunjukkan program berada pada package tersebut.
- Public class HapusDLL_2511533015
➔ Nama class utama program

2. Fungsi menghapus node awal (head)

```

Public static NodeDLL_2511533015 delHead_3015 (NodeDLL_2511533015
head_3015)

```

➔ Fungsi ini digunakan untuk menghapus node pertama pada DLL

Penjelasan per baris

- Mengecek apakah list kosong

```

if (head_3015==null) {

    return null;
}

```

}

➔ Jika head bernilai null, berarti linked list kosong. Tidak ada node yang bisa dihapus.

- Menyimpan head lama

```
NodeDLL_2511533015 temp = head_3015;
```

➔ variable temp menyimpan node head lama.

Walaupun pada program ini temp tidak dipakai lagi, biasanya digunakan untuk membantu proses penghapusan.

- Memindahkan head ke node berikutnya

```
head_3015 = head_3015.next_3015;
```

➔ head dipindahkan ke node kedua.

Contoh:

Sebelum:

1 <-> 2 <-> 3

^

Head

Sesudah:

1 <-> 2 <-> 3

^

Head

- Menghapus hubungan prev

```
if (head_3015 != null) {  
    head_3015.prev_3015 = null;  
}
```

➔ karena node pertama sudah dihapus, maka node baru tidak boleh memiliki pointer ke node lama

- Mengembalikan head baru

```
return head_3015;
```

➔ head yang baru dikembalikan

3. Fungsi menghapus node terakhir

```
public static NodeDLL_2511533015 delLast_3015 (NodeDLL_2511533015  
head_3015)
```

➔ fungsi ini digunakan untuk menghapus node terakhir.

- Jika list kosong

```
if (head_3015 == null) {
    return null;
}
```

Tidak ada node yang dapat dihapus

- Jika hanya ada satu node

```
if (head_3015.next_3015 == null) {
    return null;
}
```

Jika node hanya satu

1

- Menelusuri sampai node terakhir

```
NodeDLL_2511533015 curr_3015 = head_3015;
```

```
while (curr_3015.next_3015 != null) {
    curr_3015 = curr_3015.next_3015;
}
```

➔ loop berjalan sampai curr berada di node terakhir

Contoh:

1 <-> 2 <-> 3 <-> 4 <-> 5

^

Curr

- Memutus hubungan node terakhir

```
if (curr_3015.prev_3015 != null) {
    curr_3015.prev_3015.next_3015 = null;
}
```

➔ node sebelum node terakhir diarahkan ke null.

Sebelum:

4 <-> 5

Sesudah:

4 -> null

Node 5 terhapus dari list

- Mengembalikan head

```
return head_3015;
```

➔ karena head tidak berubah, maka head lama dikembalikan

4. Fungsi menghapus node pada posisi tertentu

```
public static NodeDLL_2511533015 delPos_3015 (NodeDLL_2511533015  
head_3015, int pos)
```

➔ fungsi ini digunakan untuk menghapus node berdasarkan posisi

Contoh:

Posisi:

1 2 3 4

A<->B<->C<->D

Jika pos = 2, maka node B dihapus

- Mengecek list kosong

```
if (head_3015 == null) {  
    return head_3015;  
}
```

➔ jika kosong, langsung kembalikan head

- Menelusuri node sesuai posisi

```
NodeDLL_2511533015 curr_3015 = head_3015;
```

```
for (int i = 1; curr_3015 != null && i < pos ; ++i) {  
    curr_3015 = curr_3015.next_3015;  
}
```

➔ loop berjalan sampai posisi yang ingin dihapus ditemukan

- Jika posisi tidak ada

```
if (curr_3015 == null) {  
    return head_3015;  
}
```

➔ misalnya posisi-10 padahal node hanya 5

- Menghubungkan node sebelum dan sesudah

=Update next

```
if (curr_3015.prev_3015 != null) {  
    curr_3015.prev_3015.next_3015 = curr_3015.next_3015;  
}
```

➔node sebelum diarahkan ke node setelahnya

=Update prev

```
if (curr_3015.next_3015 != null) {  
    curr_3015.next_3015.prev_3015 = curr_3015;  
}
```

➔node sesudah diarahkan Kembali ke node sebelumnya

- Jika yang dihapus adalah head

```
if (head_3015 == curr_3015) {  
    head_3015 = curr_3015.next_3015;  
}
```

➔jika node yang dihapus Adalah node pertama, maka head dipindahkan

5. Fungsi menampilkan linked list

```
public static void printList_3015 (NodeDLL_2511533015 head_3015)
```

➔digunakan untuk mencetak isi DLL

=proses traversal

```
NodeDLL_2511533015 curr_3015 = head_3015;
```

```
while (curr_3015 != null) {  
    System.out.print(curr_3015.data_3015 + "<-> ");  
    curr_3015 = curr_3015.next_3015;  
}
```

➔traversal dimulai dari head sampai akhir node

Contoh output:

1<-> 2<-> 3<-> 4<->

6. Method main

```
public static void main (String [] args)
```

➔bagian utama program

7. Membuat doubly linked list

```
NodeDLL_2511533015 head_3015 = new NodeDLL_2511533015 (1);
```

➔membuat node pertama berisi angka 1

- Menambahkan node berikutnya

```
head_3015.next_3015 = new NodeDLL_2511533015 (2);
```

```
head_3015.next_3015.prev_3015 = head_3015;
```

artinya:

1 <-> 2

- Next node 1 menunjuk node 2
- Prev node 2 menunjuk node 1

Langkah yang sama dilakukan sampai node 5

Hasil akhir:

1 <-> 2 <-> 3 <-> 4 <-> 5

8. Menampilkan DLL awal

```
System.out.print("DLL Awal: ");
```

```
printList_3015 (head_3015);
```

output:

DLL Awal: 1<-> 2<-> 3<-> 4<-> 5<->

9. Menghapus head

```
head_3015 = delHead_3015 (head_3015);
```

node 1 dihapus.

Hasil:

2 <-> 3 <-> 4 <-> 5

10. Menghapus node terakhir

```
head_3015 = delLast_3015 (head_3015);
```

node 5 dihapus

hasil:

2 <-> 3 <-> 4

11. Menghapus node posis ke 2

```
head_3015 = delPos_3015 (head_3015, 2);
```

posisi ke 2 adalah node 3

hasil:

2 <-> 4

C. Output

```
<terminated> HapusDLL_2511533015 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.h
DLL Awal: 1<-> 2<-> 3<-> 4<-> 5<->
Setelah head dihapus: 2<-> 3<-> 4<-> 5<->
Setelah node terakhir dihapus: 2<-> 3<-> 4<->
menghapus node ke 2: 2<-> 4<->
```

⇒ Class InsertDLL_2511533015

A. Kode program

```
package pekan6_2511533015;

public class InsertDLL_2511533015 {
    //menambahkan node di awal DLL
    static NodeDLL_2511533015 insertBegin (NodeDLL_2511533015 head_3015, int data_3015) {
        //buat node baru
        NodeDLL_2511533015 new_node_3015 = new NodeDLL_2511533015 (data_3015);
        //jadikan pointer nextnya head
        new_node_3015.next_3015 = head_3015;
        //jadikan pointer prev head ke new node
        if (head_3015 != null) {
            head_3015.prev_3015 = new_node_3015;
        }
        return new_node_3015;
    }

    //fungsi menambahkan node di akhir
    public static NodeDLL_2511533015 insertEnd (NodeDLL_2511533015 head_3015, int
newData_3015) {
        //buat node baru
        NodeDLL_2511533015 newNode_3015 = new NodeDLL_2511533015 (newData_3015);
        //jika dll null jadikan head
        if (head_3015 == null) {
            head_3015 = newNode_3015;
        }
    }
}
```

```

    }
    else {
        NodeDLL_2511533015 curr_3015 = head_3015;
        while (curr_3015.next_3015 != null) {
            curr_3015 = curr_3015.next_3015;
        }
        curr_3015.next_3015 = newNode_3015;
        newNode_3015.prev_3015 = curr_3015;
    }
    return head_3015;
}

//menambahkan node di posisi tertentu
public static NodeDLL_2511533015 insertAtPosition (NodeDLL_2511533015 head_3015, int
pos, int new_data_3015) {
    //buat node baru
    NodeDLL_2511533015 new_node_3015 = new NodeDLL_2511533015
(new_data_3015);
    if(pos == 1) {
        new_node_3015.next_3015 = head_3015;
        if (head_3015 != null) {
            head_3015.prev_3015 = new_node_3015;}
        head_3015 = new_node_3015;
        return head_3015;
    }
    NodeDLL_2511533015 curr_3015 = head_3015;
    for (int i = 1; i < pos - 1 && curr_3015 != null; ++i) {
        curr_3015 = curr_3015.next_3015;}
    if(curr_3015 == null){
        System.out.println ("Posisi tidak ada");
        return head_3015;
    }
    new_node_3015.prev_3015 = curr_3015;
    new_node_3015.next_3015 = curr_3015.next_3015;
    curr_3015.next_3015 = new_node_3015;

```



```

        if (new_node_3015.next_3015 != null) {
            new_node_3015.next_3015.prev_3015 = new_node_3015;
        }
        return head_3015;
    }

    public static void printList (NodeDLL_2511533015 head_3015) {
        NodeDLL_2511533015 curr_3015 = head_3015;
        while (curr_3015 != null) {
            System.out.print(curr_3015.data_3015 + " <->");
            curr_3015 = curr_3015.next_3015;
        }
        System.out.println();
    }

    public static void main (String[] args) {
        //membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511533015 head_3015 = new NodeDLL_2511533015(2);
        head_3015.next_3015 = new NodeDLL_2511533015 (3);
        head_3015.next_3015.prev_3015 = head_3015;
        head_3015.next_3015.next_3015 = new NodeDLL_2511533015(5);
        head_3015.next_3015.next_3015.prev_3015 = head_3015.next_3015;
        head_3015.next_3015.next_3015.prev_3015 = head_3015.next_3015;
        //cetak DLL awal
        System.out.print("DLL Awal: ");
        printList(head_3015);
        //tambah 1 di awal
        head_3015 = insertBegin (head_3015,1);
        System.out.print(
            "simpul 1 ditambah di awal: ");
        printList (head_3015);
        //tambah 6 di akhir
        System.out.print(
            "simpul 6 ditambah di akhir: ");
        int data_3015 = 6;
        head_3015 = insertEnd (head_3015, data_3015);
    }
}

```

```

    printList(head_3015);
    //menambahkan node 4 di posisi 4
    System.out.print("tambah node 4 di posisi 4: ");
    int data2_3015 = 4;
    int pos_3015 = 4;
    head_3015 = insertAtPosition(head_3015, pos_3015, data2_3015);
    printList(head_3015);
}
}

```

B. Penjelasan

- Struktur class

public class InsertDLL_2511533015

➔ class ini berisi semua method untuk manipulasi doubly linked list

1. Fungsi menambah node di awal

static NodeDLL_2511533015 insertBegin

(NodeDLL_2511533015 head_3015, int data_3015)

➔ method ini digunakan untuk menambahkan kode di bagian paling depan

- Membuat node baru

NodeDLL_2511533015 new_node_3015 =

new NodeDLL_2511533015 (data_3015);

➔ membuat node baru dengan data yang dikirim

Misalnya:

insertBegin(head, 1);

maka dibuat node baru berisi 1.

- Menghubungkan node baru ke head lama

new_node_3015.next_3015 = head_3015;

➔ pointer next dari node baru diarahkan ke head lama

Contoh:

Sebelum:

2 <-> 3 <-> 5

Sesudah:

1 -> 2 <-> 3 <-> 5

- Mengatur prev head lama

```
if (head_3015 != null) {
    head_3015.prev_3015 = new_node_3015;
}
```

➔karena node 1 sekarang berada di depan node 2, maka:

Prev node 2 harus menunjuk ke node 1.

Hasil:

1 <-> 2 <-> 3 <-> 5

- Mengembalikan head baru

```
return new_node_3015;
```

➔karena node baru berada paling depan, maka node baru menjadi head

2. Fungsi menambah node di akhir

```
public static NodeDLL_2511533015 insertEnd
```

```
(NodeDLL_2511533015 head_3015,
```

```
int newData_3015)
```

➔ method ini digunakan untuk menambahkan node di akhir linked list

- Membuat node baru

```
NodeDLL_2511533015 newNode_3015 =
new NodeDLL_2511533015 (newData_3015);
```

➔membuat node baru

- Jika DLL kosong

```
if (head_3015 == null) {
    head_3015 = newNode_3015;
}
```

➔jika linked list kosong, maka node baru langsung menjadi head

- Menelusuri sampai node terakhir

```
NodeDLL_2511533015 curr_3015 = head_3015;
```

```
while (curr_3015.next_3015 != null) {
    curr_3015 = curr_3015.next_3015;
}
```

➔loop berjalan sampai node terakhir ditemukan

Contoh:

2 <-> 3 <-> 5

^

Curr

- Menambahkan node baru di belakang
`curr_3015.next_3015 = newNode_3015;`
➔ node terakhir diarahkan ke node baru
- Menambahkan prev node baru
`newNode_3015.prev_3015 = curr_3015;`
➔ pointer prev node baru diarahkan ke node sebelumnya

Hasil akhir:

2 <-> 3 <-> 5 <-> 6

3. Fungsi menambah node di posisi tertentu

```
public static NodeDLL_2511533015 insertAtPosition  
(NodeDLL_2511533015 head_3015,  
int pos,  
int new_data_3015)
```

- Membuat node baru
`NodeDLL_2511533015 new_node_3015 =
new NodeDLL_2511533015 (new_data_3015);`
- Jika posisi = 1
`if (pos == 1)`
➔ artinya: node ditambahkan di awal
- Menjadikan node baru sebagai head
`new_node_3015.next_3015 = head_3015;`
➔ node baru diarahkan ke head lama
- Mengatur prev head lama
`if (head_3015 != null) {
 head_3015.prev_3015 = new_node_3015;
}`
- Memindahkan head
`head_3015 = new_node_3015;`
- Jika posisi bukan 1
=Menelusuri node sebelum posisi tujuan

```
NodeDLL_2511533015 curr_3015 = head_3015;
```

```
for (int i = 1;  
     i < pos - 1 && curr_3015 != null;  
     ++i) {
```

```
    curr_3015 = curr_3015.next_3015;  
}
```

=misalnya ingin menambah di posisi 4:

1 <-> 2 <-> 3 <-> 5

=loop berhenti di node 3

- Jika posisi tidak ada

```
if (curr_3015 == null){  
    System.out.println ("Posisi tidak ada");  
    return head_3015;  
}
```

=jika posisi melebihi jumlah node

- Menyisipkan node baru

=mengatur prev node baru

```
new_node_3015.prev_3015 = curr_3015;
```

=mengatur next node baru

```
new_node_3015.next_3015 =
```

```
curr_3015.next_3015;
```

=menghubungkan node sebelumnya ke node baru

```
curr_3015.next_3015 = new_node_3015;
```

=mengatur prev node sesudahnya

```
if (new_node_3015.next_3015 != null) {  
    new_node_3015.next_3015.prev_3015 =  
    new_node_3015;  
}
```

Hasil

1 <-> 2 <-> 3 <-> 4 <-> 5

4. Fungsi printlist

```
public static void printList
```

(NodeDLL_2511533015 head_3015)

➔digunakan untuk mencetak isi linked list

- Traversal

while (curr_3015 != null)

➔berjalan dari head sampai akhir

- Mencetak data

System.out.print(curr_3015.data_3015 + " <->");

=contoh output:

1 <->2 <->3 <->

5. Method main

public static void main (String[] args)

➔bagian utama program

- Membuat DLL awal

2 <-> 3 <-> 5

➔dibuat manual menggunakan pointer next dan prev

- Menampilkan DLL awal

System.out.print("DLL Awal: ");

printList(head_3015);

=output:

DLL Awal: 2 <->3 <->5 <->

- Menambah node 1 di awal

head_3015 = insertBegin(head_3015,1);

=hasil:

1 <-> 2 <-> 3 <-> 5

- Menambah node 6 di akhir

head_3015 = insertEnd(head_3015,6);

=hasil:

1 <-> 2 <-> 3 <-> 5 <-> 6

- Menambah node 4 di posisi 4

head_3015 =

insertAtPosition(head_3015,4,4);

➔node 4 disisipkan sebelum node 5

=hasil:

1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6

- Alur perubahan linked list

=awal:

2 <-> 3 <-> 5

=setelah tambah 1 di awal

1 <-> 2 <-> 3 <-> 5

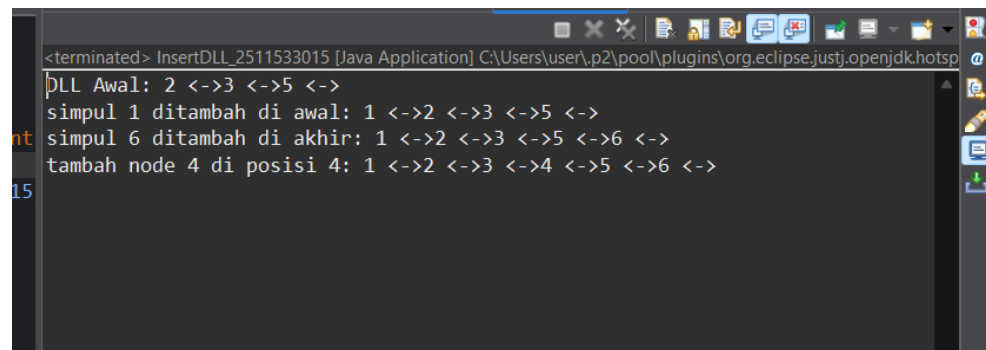
=setelah tambah 6 di akhir

1 <-> 2 <-> 3 <-> 5 <-> 6

=setelah tambah 4 di posisi 4

1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6

C. Output



```
<terminated> InsertDLL_2511533015 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp
DLL Awal: 2 <->3 <->5 <->
simpul 1 ditambah di awal: 1 <->2 <->3 <->5 <->
simpul 6 ditambah di akhir: 1 <->2 <->3 <->5 <->6 <->
tambah node 4 di posisi 4: 1 <->2 <->3 <->4 <->5 <->6 <->
```

⇒ Class NodeDLL_2511533015

A. Kode program

```
package pekan6_2511533015;

public class NodeDLL_2511533015 {
    //mendefinisikan kelas Node
    int data_3015; // data
    NodeDLL_2511533015 next_3015; // pointer ke next node
    NodeDLL_2511533015 prev_3015; // pointer ke previous node

    //konstruktor
    public NodeDLL_2511533015(int data_3015) {
        this.data_3015 = data_3015;
        this.next_3015 = null;
        this.prev_3015 = null;
    }
}
```

B. Penjelasan

1. penjelasan program

```
package pekan6_2511533015;
```

➔ baris ini menunjukkan class berada di dalam package:

```
pekan6_2511533015
```

➔ package digunakan untuk mengelompokkan file java agar lebih rapi dan terorganisir

2. deklarasi class node

```
public class NodeDLL_2511533015 {
```

=class ini Bernama:

```
NodeDLL_2511533015
```

➔ class ini digunakan untuk merepresentasikan 1 node pada doubly linked list

3. atribut pada node

a. data

```
int data_3015;
```

➔ variabel ini digunakan untuk menyimpan nilai/ data pada node

=contoh:

```
[5]
```

=maka:

```
Data_3015 = 5
```

b. pointer next

```
NodeDLL_2511533015 next_3015;
```

➔ variabel ini menyimpan alamat node berikutnya

=contoh:

```
1 <-> 2
```

=pada node 1:

Next_3015 menunjuk ke node 2

c. pointer previous

```
NodeDLL_2511533015 prev_3015;
```

➔ variable ini menyimpan alamat node sebelumnya:

=contoh:

```
1 <-> 2
```

=pada node 2:

Prev_3015 menunjuk ke node 1

4. constructor node

```
public NodeDLL_2511533015 (int data_3015)
```

➔ constructor digunakan untuk membuat object node baru

=Ketika dipanggil:

```
new NodeDLL_2511533015(10);
```

=maka dibuat node dengan data 10

5. isi constructor

- menyimpan data

```
this.data_3015 = data_3015;
```

➔ data yang dikirim ke constructor disimpan ke variable node

=misalnya:

```
new NodeDLL_2511533015(7);
```

=maka:

```
data_3015 = 7
```

6. pointer next diisi null

```
this.next_3015 = null;
```

➔ karena node baru belum terhubung ke node lain, maka pointer next bernilai null

7. pointer prev diisi null

```
this.prev_3015 = null;
```

➔ karena belum memiliki node sebelumnya, maka prev juga null

8. contoh pembuatan node

```
NodeDLL_2511533015 node1 =
```

```
new NodeDLL_2511533015(5);
```

=hasil isi node:

```
prev = null
```

```
data = 5
```

```
next = null
```

=ilustrasi

```
null <- [5] -> null
```

9. contoh menghubungkan node

=misalnya:

```
NodeDLL_2511533015 node1 =
```

```
new NodeDLL_2511533015(1);
```

```
NodeDLL_2511533015 node2 =  
new NodeDLL_2511533015(2);
```

```
node1.next_3015 = node2;
```

```
node2.prev_3015 = node1;
```

=maka terbentuk:

```
null <- [1] <-> [2] -> null
```

=penjelasan:

- node1.next_3015 = node2
→ node 1 menunjuk ke node 2
- node2.prev_3015 = node1
→ node 2 menunjuk Kembali ke node 1

10. fungsi class NodeDLL

class ini digunakan sebagai:

- a. pembentuk node
- b. penyimpan data
- c. penghubung antar node

tanpa class ini, program insert dan delete pada doubly linked list tidak dapat berjalan

⇒ class PenelusuranDLL_2511533015

A. kode program

```
package pekan6_2511533015;  
  
public class PenelusuranDLL_2511533015 {  
    //fungsi penelusuran maju  
    static void forwardTraversal (NodeDLL_2511533015 head_3015) {  
        //memulai penelusuran dari head  
        NodeDLL_2511533015 curr_3015 = head_3015;  
        //lanjutkan sampai akhir  
        while (curr_3015 != null) {  
            //print data  
            System.out.print (curr_3015.data_3015 + " <-> ");  
            //pindah ke node berikutnya  
            curr_3015 = curr_3015.next_3015;  
        }  
        //print spasi  
        System.out.println ();  
    }  
    //fungsi penelusuran mundur
```

```

static void backwardTraversal (NodeDLL_2511533015 tail) {
    //mulai dari akhir
    NodeDLL_2511533015 curr_3015 = tail;
    //lanjut sampai head
    while (curr_3015 != null) {
        //cetak data
        System.out.print (curr_3015.data_3015 + " <-> ");
        //pindah ke node sebelumnya
        curr_3015 = curr_3015.prev_3015;
    }
    //cetak spasi
    System.out.println ();
}

public static void main (String [] args) {
    //cetak DLL
    NodeDLL_2511533015 head_3015 = new
NodeDLL_2511533015 (1);
    NodeDLL_2511533015 second_3015 = new
NodeDLL_2511533015 (2);
    NodeDLL_2511533015 third_3015 = new
NodeDLL_2511533015 (3);

    head_3015.next_3015 = second_3015;
    second_3015.prev_3015 = head_3015;
    second_3015.next_3015 = third_3015;
    third_3015.prev_3015 = second_3015;

    System.out.println ("Penelusuran maju: ");
    forwardTraversal (head_3015);

    System.out.println ("Penelusuran mundur: ");
    backwardTraversal (third_3015);
}
}

```

B. penjelasan

struktur program

public class PenelusuranDLL_2511533015

class ini berisi:

- method traversal maju
- method traversal mundur
- method main
 1. fungsi penelusuran maju

static void forwardTraversal

(NodeDLL_2511533015 head_3015)

➔method ini digunakan untuk menelusuri linked list dari:

Head➔tail

a. membuat pointer bantu

NodeDLL_2511533015 curr_3015 = head_3015;

➔variabel curr_3015 digunakan untuk berjalan dari node ke node

Awalnya dimulai dari head

b. melakukan traversal

while (curr_3015 != null)

➔loop berjalan selama node masih ada

Jika sudah mencapai akhir linked list:

- curr_3015 = null

- loop berhenti

c. mencetak data

System.out.print

(curr_3015.data_3015 + " <-> ");

➔menampilkan isi node saat ini.

d. Pindah ke node berikutnya

curr_3015 = curr_3015.next_3015;

➔karena traversal maju, maka menggunakan pointer next

e. Ilustrasi traversal maju

=Linked list:

1 <-> 2 <-> 3

=Alur traversal:

curr = 1

curr = 2

curr = 3

curr = null

=output:

1 <-> 2 <-> 3 <->

2. Fungsi penelusuran mundur

static void backwardTraversal

(NodeDLL_2511533015 tail)

➔method ini digunakan untuk traversal dari:

- Tail → head
 - a. Memulai dari tail


```
NodeDLL_2511533015 curr_3015 = tail;
```

 → traversal dimulai dari node terakhir
 - b. Loop traversal


```
while (curr_3015 != null)
```

 → loop berjalan selama node masih ada
 - c. Mencetak data


```
System.out.print  
(curr_3015.data_3015 + " <-> ");
```

 → menampilkan isi node
 - d. Bergerak ke node sebelumnya


```
curr_3015 = curr_3015.prev_3015;
```

 → karena traversal mundur, maka menggunakan pointer prev
 - e. Ilustrasi traversal mundur
 - a. Linked list


```
1 <-> 2 <-> 3
```

 Traversal dimulai dari node 3.
 - b. Alur


```
curr = 3  
curr = 2  
curr = 1  
curr = null
```
 - c. Output:


```
3 <-> 2 <-> 1 <->
```
- 3. Method main


```
public static void main (String [] args)
```

 → bagian utama program
- Membuat node


```
NodeDLL_2511533015 head_3015 =  
new NodeDLL_2511533015 (1);
```



```
NodeDLL_2511533015 second_3015 =  
new NodeDLL_2511533015 (2);
```

```
NodeDLL_2511533015 third_3015 =  
new NodeDLL_2511533015 (3);
```

- a. Membuat 3 node:

Node 1

Node 2

Node 3

- Menghubungkan node

- a. Menghubungkan node 1 ke node 2

```
head_3015.next_3015 = second_3015;
```

```
second_3015.prev_3015 = head_3015;
```

=hasil:

1 <-> 2

- b. Menghubungkan node 2 ke node 3

```
second_3015.next_3015 = third_3015;
```

```
third_3015.prev_3015 = second_3015;
```

hasil akhir:

1 <-> 2 <-> 3

- c. Menjalankan traversal maju

```
forwardTraversal (head_3015);
```

➔traversal dimulai dari head (1)

=output:

1 <-> 2 <-> 3 <->

- d. Menjalankan traversal mundur

```
backwardTraversal (third_3015);
```

➔traversal dimulai dari tail (3)

=output:

3 <-> 2 <-> 1 <->

- e. Alur program

- a. DLL yang dibuat

1 <-> 2 <-> 3

- b. Forward traversal

Menggunakan:

next_3015

arah:

1 → 2 → 3

c. Backward traversal

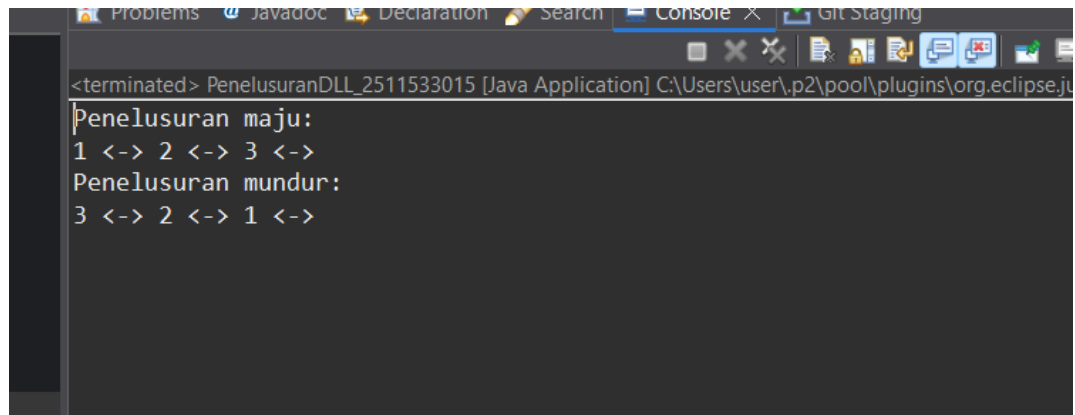
Menggunakan:

prev_3015

arah:

3 → 2 → 1

C. output.



```
<terminated> PenelusuranDLL_2511533015 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.ju
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

BAB III

PENUTUP

3.1 Ringkasan hasil praktikum

Berdasarkan praktikum yang telah dilakukan, diperoleh hasil bahwa Doubly Linked List dapat digunakan untuk mengelola data secara dinamis dengan lebih fleksibel dibandingkan array. Praktikum ini berhasil mengimplementasikan beberapa operasi dasar pada Doubly Linked List menggunakan bahasa pemrograman Java.

Pada class `NodeDLL_2511533015`, dibuat struktur node yang memiliki data, pointer next, dan pointer prev sebagai dasar pembentukan linked list. Selanjutnya, pada class `InsertDLL_2511533015` berhasil dilakukan operasi penambahan node di awal, di akhir, dan pada posisi tertentu. Program juga berhasil menampilkan perubahan struktur linked list setelah proses insert dilakukan.

Pada class `HapusDLL_2511533015`, program berhasil melakukan penghapusan node di awal, di akhir, dan pada posisi tertentu dengan melakukan pembaruan pointer next dan prev secara benar sehingga struktur linked list tetap terhubung dengan baik.

Selain itu, pada class `PenelusuranDLL_2511533015`, program berhasil melakukan traversal maju menggunakan pointer next dan traversal mundur menggunakan pointer prev. Hal ini menunjukkan bahwa Doubly Linked List memungkinkan proses penelusuran data dilakukan dari dua arah.

Secara keseluruhan, praktikum ini memberikan pemahaman mengenai konsep dasar Doubly Linked List, cara kerja pointer antar node, serta implementasi operasi dasar linked list dalam bahasa Java.

3.2 Saran pengembangan

Adapun saran pengembangan dari praktikum ini adalah sebagai berikut:

1. Program dapat dikembangkan menggunakan input dari user agar lebih interaktif.
2. Dapat ditambahkan menu program menggunakan perulangan dan percabangan sehingga pengguna dapat memilih operasi yang diinginkan.
3. Program dapat dikembangkan menggunakan konsep object oriented programming (OOP) yang lebih lengkap.
4. Dapat ditambahkan fitur pencarian data (searching) pada linked list.
5. Program dapat dikembangkan menjadi aplikasi sederhana seperti playlist musik atau sistem antrean.
6. Dapat ditambahkan validasi input untuk menghindari kesalahan saat proses insert dan delete.
7. Tampilan output program dapat dibuat lebih menarik dan terstruktur agar mudah dipahami pengguna.

DAFTAR PUSTAKA

1. Data Structures and Algorithms in Java. Goodrich, Michael T., Roberto Tamassia, dan Michael H. Goldwasser. *Data Structures and Algorithms in Java*. 6th Edition. Wiley, 2014.
2. Fundamentals of Data Structures in C. Horowitz, Ellis dan Sartaj Sahni. *Fundamentals of Data Structures in C*. Silicon Press, 2008.
3. Java: How to Program. Deitel, Paul dan Harvey Deitel. *Java: How to Program*. 11th Edition. Pearson Education, 2018.
4. Introduction to Algorithms. Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, dan Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009.
5. [Oracle Java Documentation](#)
6. [GeeksforGeeks - Doubly Linked List in Java](#)
7. [Programiz - Doubly Linked List Data Structure](#)

TUGAS

a. Class= lagu_2511533015

1. Kode program

```
package tugaspekan6_2511533015;

//kelas node lagu
public class Lagu_2511533015 {
    //atribut lagu
    private String judul_3015;
    private String penyanyi_3015;

    //pointer next dan prev
    Lagu_2511533015 next_3015;
    Lagu_2511533015 prev_3015;

    //constructor
    public Lagu_2511533015 (String judul_3015, String penyanyi_3015) {
        this.judul_3015 = judul_3015;
        this.penyanyi_3015 = penyanyi_3015;
        this.next_3015 = null;
        this.prev_3015 = null;
    }

    //getter judul
    public String getJudul_3015() {
        return judul_3015;
    }

    //getter penyanyi
    public String getPenyanyi_3015() {
        return penyanyi_3015;
    }

    //setter penyanyi
    public void setPenyanyi_3015 (String penyanyi_3015) {
        this.penyanyi_3015 = penyanyi_3015;
    }
}
```

2. Penjelasan

1. Package

```
package tugaspekan6_2511533015;
```

➔baris ini menunjukkan bahwa class berada di dalam package:

tugaspekan6_2511533015

=package digunakan untuk:

- Mengelompokkan file java
- Membuat program lebih rapi
- Mempermudah pengelolaan class

2. Deklarasi class

```
public class Lagu_2511533015
```

➔class ini Bernama:

Lagu_2511533015

Class ini digunakan untuk merepresentasikan 1 node lagu dalam playlist

Artinya:

- Satu object = satu lagu

3. Atribut lagu

a. Judul lagu

```
private String judul_3015;
```

variable ini digunakan untuk menyimpan:

- Judul lagu
=Contoh:
“hati hati di jalan”
=Keyword private berarti:
- Data hanya bisa diakses dari dalam class
- Lebih aman
- Menerapkan konsep encapsulation

b. Nama penyanyi

```
private String penyanyi_3015;
```

variable ini digunakan untuk menyimpan:

- Nama penyanyi lagu
=contoh:
“tulus”

4. Pointer next dan prev

a. Pointer next

```
Lagu_2511533015 next_3015;
```

➔pointer next_3015 digunakan untuk menunjuk:

- Lagu berikutnya

Contoh:

Lagu A <-> Lagu B

Maka:

=next pada lagu A menunjuk ke lagu B

b. Pointer prev

```
Lagu_2511533015 prev_3015;
```

➔pointer prev_3015 digunakan untuk menunjuk:

- Lagu sebelumnya

Contoh:

Lagu A <-> Lagu B

Maka:

=prev pada lagu B menunjuk ke lagu A

5. Constructor

```
public Lagu_2511533015
```

```
(String judul_3015,
```

```
String penyanyi_3015)
```

➔constructor digunakan untuk:

- Membuat object lagu baru

=Ketika dipanggil:

```
new Lagu_2511533015(
```

```
    "Komang",
```

```
    "Raim Laode"
```

```
);
```

=maka akan dibuat node lagu baru

6. Isi constructor

a. Menyimpan judul lagu

```
this.judul_3015 = judul_3015;
```

artinya:

- Data judul yang dikirim disimpin ke atribut class

Contoh:

"Komang"

Disimpan ke:

judul_3015

b. Menyimpan nama penyanyi

```
this.penyanyi_3015 = penyanyi_3015;
```

artinya:

- Nama penyanyi disimpan ke atribut class

c. Pointer next diisi null

```
this.next_3015 = null;
```

➔ karena node baru belum terhubung ke node lain, maka:

- Next = null

d. Pointer prev diisi null

```
this.prev_3015 = null;
```

➔ karena node belum memiliki node sebelumnya:

- Prev = null

7. Getter judul

```
public String getJudul_3015()
```

method getter digunakan untuk:

- Mengambil nilai judul lagu

a. Mengembalikan judul

```
return judul_3015;
```

contoh:

```
lagu.getJudul_3015();
```

output:

Komang

8. Getter penyanyi

```
public String getPenyanyi_3015()
```

➔method ini digunakan untuk:

Mengambil nama penyanyi

a. Mengembalikan penyanyi

return penyanyi_3015;

=contoh:

lagu.getPenyanyi_3015();

=output:

Raim Laode

9. Setter penyanyi

public void setPenyanyi_3015

(String penyanyi_3015)

➔ method setter digunakan untuk:

Mengubah data penyanyi

a. Mengubah nilai penyanyi

this.penyanyi_3015 = penyanyi_3015;

contoh:

lagu.setPenyanyi_3015("Tulus");

maka:

data penyanyi berubah menjadi “tulus”

10. Contoh pembuatan lagu

Lagu_2511533015 lagu1 =

new Lagu_2511533015(

"Komang",

"Raim Laode"

);

=isi node:

judul = Komang

penyanyi = Raim Laode

next = null

prev = null

11. Contoh menghubungkan lagu

```
Lagu_2511533015 lagu1 =
new Lagu_2511533015(
    "Komang",
    "Raim Laode"
);
```

```
Lagu_2511533015 lagu2 =
new Lagu_2511533015(
    "Monokrom",
    "Tulus"
);
```

```
lagu1.next_3015 = lagu2;
lagu2.prev_3015 = lagu1;
=hasil:
```

Komang <-> Monokrom

b. Class=Musik_2511533015

1. Kode program

```
package tugaspekan6_2511533015;

public class Musik_2511533015 {

    // head dan tail playlist
    Lagu_2511533015 head_3015 = null;
    Lagu_2511533015 tail_3015 = null;

    // 1. tambah lagu di akhir playlist
    public void tambahLagu_3015(String judul_3015,
                                String penyanyi_3015) {

        // membuat node lagu baru
        Lagu_2511533015 laguBaru_3015 =
            new Lagu_2511533015(
                judul_3015,
                penyanyi_3015
```



```

    );

    // jika playlist kosong
    if (head_3015 == null) {

        head_3015 = laguBaru_3015;
        tail_3015 = laguBaru_3015;

    } else {

        // hubungkan node terakhir dengan lagu baru
        tail_3015.next_3015 = laguBaru_3015;
        laguBaru_3015.prev_3015 = tail_3015;

        // pindahkan tail ke lagu baru
        tail_3015 = laguBaru_3015;
    }

    System.out.println(
        "Lagu \"" + judul_3015 +
        "\" berhasil ditambahkan"
    );
}

// 2. hapus lagu pertama
public void hapusLaguAwal_3015() {

    // jika playlist kosong
    if (head_3015 == null) {
        System.out.println("Playlist kosong");
        return;
    }

    // jika hanya ada satu lagu
    if (head_3015 == tail_3015) {

        head_3015 = null;
        tail_3015 = null;

    } else {

        // pindahkan head ke node berikutnya
        head_3015 = head_3015.next_3015;

        // prev head baru harus null
        head_3015.prev_3015 = null;
    }

    System.out.println(
        "Lagu awal berhasil dihapus"
    );
}

```

```

    );
}

// 3. hapus lagu terakhir
public void hapusLaguAkhir_3015() {

    // jika playlist kosong
    if (tail_3015 == null) {
        System.out.println("Playlist kosong");
        return;
    }

    // jika hanya ada satu lagu
    if (head_3015 == tail_3015) {

        head_3015 = null;
        tail_3015 = null;

    } else {

        // pindahkan tail ke node sebelumnya
        tail_3015 = tail_3015.prev_3015;

        // next tail baru harus null
        tail_3015.next_3015 = null;
    }

    System.out.println(
        "Lagu akhir berhasil dihapus"
    );
}

// 4. tampil playlist dari depan ke belakang
public void tampilMaju_3015() {

    // jika kosong
    if (head_3015 == null) {
        System.out.println("Playlist kosong");
        return;
    }

    Lagu_2511533015 curr_3015 = head_3015;

    System.out.println("Playlist Maju:");

    while (curr_3015 != null) {

        System.out.println(
            curr_3015.getJudul_3015()
            + " - "

```

```

        + curr_3015.getPenyanyi_3015()
    );

    curr_3015 = curr_3015.next_3015;
}
}

// 5. tampil playlist dari belakang ke depan
public void tampilMundur_3015() {

    // jika kosong
    if (tail_3015 == null) {
        System.out.println("Playlist kosong");
        return;
    }

    Lagu_2511533015 curr_3015 = tail_3015;

    System.out.println("Playlist Mundur:");

    while (curr_3015 != null) {

        System.out.println(
            curr_3015.getJudul_3015()
            + " - "
            + curr_3015.getPenyanyi_3015()
        );

        curr_3015 = curr_3015.prev_3015;
    }
}

// main program
public static void main(String[] args) {

    // membuat object playlist
    Musik_2511533015 playlist_3015 =
        new Musik_2511533015();

    // menambah lagu
    playlist_3015.tambahLagu_3015(
        "Hati-Hati di Jalan",
        "Tulus"
    );

    playlist_3015.tambahLagu_3015(
        "Melukis Senja",
        "Budi Doremi"
    );
}

```

```

        playlist_3015.tambahLagu_3015(
            "Komang",
            "Raim Laode"
        );

        System.out.println();

        // tampil maju
        playlist_3015.tampilMaju_3015();

        System.out.println();

        // tampil mundur
        playlist_3015.tampilMundur_3015();

        System.out.println();

        // hapus lagu awal
        playlist_3015.hapusLaguAwal_3015();

        System.out.println();

        // tampil playlist
        playlist_3015.tampilMaju_3015();

        System.out.println();

        // hapus lagu akhir
        playlist_3015.hapusLaguAkhir_3015();

        System.out.println();

        // tampil playlist
        playlist_3015.tampilMaju_3015();
    }
}

```

2. Penjelasan

1. Package

package tugaspekan6_2511533015;

➔ baris ini menunjukkan bahwa class berada pada package:

tugaspekan6_2511533015

package digunakan untuk:

- Mengelompokkan digunakan untuk:

Mengelompokkan file java
Membuat project lebih rapi
Mempermudah pengelolaan class

2. Deklarasi class

```
public class Musik_2511533015
```

class ini Bernama:

```
Musik_2511533015
```

Class ini berfungsi untuk:

- Mengelola playlist lagu
- Melakukan operasi pada doubly linked list

3. Head dan tail

```
Lagu_2511533015 head_3015 = null;
```

```
Lagu_2511533015 tail_3015 = null;
```

Program menggunakan dua pointer penting:

a. Head

Head_3015 menunjuk:

- Node pertama pada playlist

b. Tail

Tail_3015 menunjuk:

- Node terakhir pada playlist

Awalnya:

```
head = null
```

```
tail = null
```

karena playlist masih kosong

4. Method tambah lagu

```
public void tambahLagu_3015(
```

```
String judul_3015,
```

```
String penyanyi_3015)
```

=method ini digunakan untuk:

- Menambahkan lagu baru di akhir playlist

a. Membuat node baru

```
Lagu_2511533015 laguBaru_3015 =
new Lagu_2511533015(
    judul_3015,
    penyanyi_3015
);
```

=membuat objek lgu abru

Contoh:

```
tambahLagu_3015(
    "Komang",
    "Raim Laode"
);
```

Maka dibuat node:

```
judul = Komang
penyanyi = Raim Laode
```

b. Jika playlist kosong

```
if (head_3015 == null)
```

artinya:

belum ada lagu di playlist

- Menjadikan lagu baru sebagai head dan tail

```
head_3015 = laguBaru_3015;
```

```
tail_3015 = laguBaru_3015;
```

=karena hanya ada satu node::

```
head → laguBaru ← tail
```

c. Jika playlist tidak kosong

Else

Artinya:

=Sudah ada lagu sebelumnya

- Menghubungkan tail lama ke lagu baru

```
tail_3015.next_3015 = laguBaru_3015;
```

=node trakhir diarahkan ke node baru

Contoh:

A <-> B

Setelah tambah C:

B.next = C

laguBaru_3015.prev_3015 = tail_3015;

➔ node baru diarahkan ke node sebelumnya

- Memindahkan tail

tail_3015 = laguBaru_3015;

➔ karena lagu baru sekarang berada di akhir playlist

d. Output

```
System.out.println(  
    "Lagu \'" + judul_3015 +  
    "\' berhasil ditambahkan"  
);
```

e. Method hapus lagu awal

```
public void hapusLaguAwal_3015()
```

1. Jika playlist kosong

```
if (head_3015 == null)
```

2. jika hanya ada satu lagu

```
if (head_3015 == tail_3015)
```

3. Mengosongkan playlisy

```
head_3015 = null;
```

```
tail_3015 = null;
```

4. Jika lagu lebih dari satu

```
head_3015 = head_3015.next_3015;
```

5. Menghapus pointer prev

```
head_3015.prev_3015 = null;
```

f. Method hapus lagu akhir

```
public void hapusLaguAkhir_3015()
```

1. Jika playlist kosong

```
if (tail_3015 == null)
```

2. Jika hanya satu lagu

```

        if (head_3015 == tail_3015)
3. Jika lagu lebih dari satu
        tail_3015 = tail_3015.prev_3015;
g. Method tampil maju
    public void tampilMaju_3015()
    1. Traversal dimulai dari head
        Lagu_2511533015 curr_3015 = head_3015;
    2. Loop traversal
        while (curr_3015 != null)
    3. Menampilkan lagu
        System.out.println(
            curr_3015.getJudul_3015()
            + " - "
            + curr_3015.getPenyanyi_3015()
        );
    4. Bergerak ke node berikutnya
        curr_3015 = curr_3015.next_3015;
h. Method tampil mundur
    public void tampilMundur_3015()
    a. Traversal dimulai dari tell
        Lagu_2511533015 curr_3015 = tail_3015;

```


- b. Bergerak menggunakan prev
curr_3015 = curr_3015.prev_3015;
- i. Method main
public static void main(String[] args)
- j. Membuat object playlist
Musik_2511533015 playlist_3015 =
new Musik_2511533015();
- k. Menambahkan lagu
playlist_3015.tambahLagu_3015(
"Hati-Hati di Jalan",
"Tulus"
);
- l. Menampilkan playlist maju
playlist_3015.tampilMaju_3015();
- m. Menampilkan playlist mundur
playlist_3015.tampilMundur_3015();
- n. Menghapus lagu awal
playlist_3015.hapusLaguAwal_3015();
- o. Menghapus lagu akhir
playlist_3015.hapusLaguAkhir_3015();
- c. Output

Lagu "Hati-Hati di Jalan" berhasil ditambahkan
Lagu "Melukis Senja" berhasil ditambahkan
Lagu "Komang" berhasil ditambahkan

Playlist Maju:

Hati-Hati di Jalan - Tulus
Melukis Senja - Budi Doremi
Komang - Raim Laode

Playlist Mundur:|

Komang - Raim Laode
Melukis Senja - Budi Doremi
Hati-Hati di Jalan - Tulus

Lagu awal berhasil dihapus

Playlist Maju:

Melukis Senja - Budi Doremi
Komang - Raim Laode

Lagu akhir berhasil dihapus

Playlist Maju:

Melukis Senja - Budi Doremi